```
''****************************
''*  TV Driver v1.0          *
''*  (C) 2004 Parallax, Inc. *
''****************************

CON

  fntsc         = 3_579_545     'NTSC color frequency
  lntsc         = 3640          'NTSC color cycles per line * 16
  sntsc         = 624           'NTSC color cycles per sync * 16

  fpal          = 4_433_618     'PAL color frequency
  lpal          = 4540          'PAL color cycles per line * 16
  spal          = 848           'PAL color cycles per sync * 16

  'paramcount   = 14
  paramcount    = 16

  ' Taken from COP_DRV_005
  pix_ntsc      = 10            '(11 pulses per pixel) changed to 10 gives 2.2us extra on Left, and 2.2us extra on Right
  frame_ntsc    = pix_ntsc*4    '4 pixels per frame)

PUB start(tvptr)

  cognew(@entry,tvptr)


DAT

'*******************************
'* Assembly language TV driver *
'*******************************

                        org
'
'
' Entry
'
entry                   mov     taskptr,#tasks          'reset tasks

' RGW TODO: reduce this as color load is no longer needed here.
                        mov     x,#10                   'perform task sections initially
:init                   jmpret  taskret,taskptr
                        djnz    x,#:init
'
'
' Superfield
'
superfield              mov     taskptr,#tasks          'reset tasks

                        test    _mode,#%0001    wc      'if ntsc, set phaseflip
       if_nc            mov     phaseflip,phasemask

                        test    _mode,#%0010    wz      'get interlace into z
'
'
' Field
'
                        ' Send a 'prepare scanline 0' in advance
                        mov next_line, #0
                        wrlong Next_Line, _DoLines


field                   mov     x,vinv                  'do invisible back porch lines
```

```
                        ' Loop for 256 pixels fetching them off scanline buffer, 4 at a time
                        mov x, #64
                        mov screen, _ScanLine
:nextpixel
                        mov tile, phaseflip
                        rdlong pixels, screen
                        xor tile, pixels
                        add screen, #4
                        waitvid tile, #%%3210
                        djnz x, #:nextpixel

                        mov     vscl,hf                 'do visible front porch pixels
                        mov     tile,phaseflip
                        xor     tile, _BackColor
                        waitvid tile, #0

                        add next_line, #1
                        wrlong Next_Line, _DoLines

                        djnz    y, #:line               'next scanline


        if_z            xor     interlace,#1    wz      'get interlace and field1 into z

                        test    _mode,#%0001    wc      'do visible front porch lines
                        mov     x,vf
        if_nz_and_c     add     x,#1
                        call    #blank_lines

        if_nz           wrlong  invisible,par           'unless interlace and field1, set status to invisible

        if_z_eq_c       call    #hsync                  'if required, do short line
        if_z_eq_c       mov     vscl,hrest
        if_z_eq_c       waitvid burst,sync_high2
        if_z_eq_c       xor     phaseflip,phasemask

                        call    #vsync_high             'do high vsync pulses

                        movs    vsync1,#sync_low1       'do low vsync pulses
                        movs    vsync2,#sync_low2
                        call    #vsync_low

                        call    #vsync_high             'do high vsync pulses

        if_nz           mov     vscl,hhalf              'if odd frame, do half line
        if_nz           waitvid burst,sync_high2

        if_z            jmp     #field                  'if interlace and field1, display field2
                        jmp     #superfield             'else, new superfield
'
'
' Blank lines
'
blank_lines             call    #hsync                  'do hsync

                        xor     tile, _BackColor        'do background
                        waitvid tile,#0

                        djnz    x,#blank_lines

blank_lines_ret         ret
'
'
' Horizontal sync
```

```
vsync_low               mov     x,vrep

vsyncx                  mov     vscl,sync_scale1
vsync1                  waitvid burst,sync_high1

                        mov     vscl,sync_scale2
vsync2                  waitvid burst,sync_high2

                        djnz    x,#vsyncx
vsync_low_ret
vsync_high_ret          ret
'
' Tasks - performed in sections during invisible back porch lines
'
tasks                   mov     t1,par                  'load parameters
                        movd    :par,#_enable           '(skip _status)
                        mov     t2,#paramcount - 1
:load                   add     t1,#4
:par                    rdlong  0,t1
                        add     :par,d0
                        djnz    t2,#:load               '+119

                        mov     t1,_pins                'set video pins and directions
                        test    t1,#$08         wc
        if_nc           mov     t2,pins0
        if_c            mov     t2,pins1
                        test    t1,#$40         wc
                        shr     t1,#1
                        shl     t1,#3
                        shr     t2,t1
                        movs    vcfg,t2
                        shr     t1,#6
                        movd    vcfg,t1
                        shl     t1,#3
                        and     t2,#$FF
                        shl     t2,t1
        if_nc           mov     dira,t2
        if_nc           mov     dirb,#0
        if_c            mov     dira,#0
        if_c            mov     dirb,t2                 '+18

                        tjz     _enable,#disabled       '+2, disabled?

                        jmpret  taskptr,taskret         '+1=140, break and return later

                        movs    :rd,#wtab               'load ntsc/pal metrics from word table
                        movd    :wr,#hvis
                        mov     t1,#wtabx - wtab
                        test    _mode,#%0001    wc
:rd                     mov     t2,0
                        add     :rd,#1
        if_nc           shl     t2,#16
                        shr     t2,#16
:wr                     mov     0,t2
                        add     :wr,d0
                        djnz    t1,#:rd                 '+54

        if_nc           movs    :ltab,#ltab             'load ntsc/pal metrics from long table
        if_c            movs    :ltab,#ltab+1
                        movd    :ltab,#fcolor
                        mov     t1,#(ltabx - ltab) >> 1
:ltab                   mov     0,0
```

```
        if_nc           shl     t2,#1
                        mov     frqa,t2                 '+147

                        jmpret  taskptr,taskret         '+1=148, break and return later

                        mov     t1,_broadcast           'set ctrb pll to _broadcast
                        mov     t2,#0                   'if 0, turn off ctrb
                        tjz     t1,:off
                        min     t1,m8                   'limit from 8MHz to 128MHz
                        max     t1,m128
                        mov     t2,#%00001_100          'adjust _broadcast to be within 4MHz-8MHz
:scale                  shr     t1,#1                   '(vco will be within 64MHz-128MHz)
                        cmp     m8,t1           wc
        if_c            add     t2,#%00000_001
        if_c            jmp     #:scale
:off                    movi    ctrb,t2
                        call    #divide
                        mov     frqb,t2                 '+165

                        jmpret  taskptr,taskret         '+1=166, break and return later

                        mov     t1,#%10100_000          'set video configuration
                        test    _pins,#$01      wc      '(swap broadcast/baseband output bits?)
        if_c            or      t1,#%01000_000
                        test    _mode,#%1000    wc      '(strip chroma from broadcast?)
        if_nc           or      t1,#%00010_000
                        test    _mode,#%0100    wc      '(strip chroma from baseband?)
        if_nc           or      t1,#%00001_000
                        and     _auralcog,#%111         '(set aural cog)
                        or      t1,_auralcog
                        movi    vcfg,t1                 '+10

                        mov     hx,_hx                  'compute horizontal metrics
                        shl     hx,#8
                        or      hx,_hx
                        shl     hx,#4

                        mov     hc2x,_ht
                        shl     hc2x,#1

                        mov     t1,_ht
                        mov     t2,_hx
                        call    #multiply
                        mov     hf,hvis
                        sub     hf,t1
                        shr     hf,#1           wc
                        mov     hb,_ho
                        addx    hb,hf
                        sub     hf,_ho                  '+52

                        mov     t1,_vt                  'compute vertical metrics
                        mov     t2,_vx
                        call    #multiply
                        test    _mode,#%0010    wc      '(if interlace, halve lines)
        if_c            shr     t1,#1
                        mov     vf,vvis
                        sub     vf,t1
                        shr     vf,#1           wc
                        neg     vb,_vo
                        addx    vb,vf
                        add     vf,_vo                  '+48

                        xor     _mode,#%0010            '+1, flip interlace bit for display
```

```
'
'
' Multiply t1 * t2 * 16 (t1, t2 = bytes)
'
multiply                shl     t2,#8+4-1

                        mov     m1,#8
:loop                   shr     t1,#1           wc
        if_c            add     t1,t2
                        djnz    m1,#:loop

multiply_ret            ret                             '+37
'
'
' Disabled - reset status, nap ~4ms, try again
'
disabled                mov     ctra,#0                 'reset ctra
                        mov     ctrb,#0                 'reset ctrb
                        mov     vcfg,#0                 'reset video

                        wrlong  outa,par                'set status to disabled

                        rdlong  t1,#0                   'get CLKFREQ
                        shr     t1,#8                   'nap for ~4ms
                        min     t1,#3
                        add     t1,cnt
                        waitcnt t1,#0

                        jmp     #entry                  'reload parameters



{
'+Debug = Discard after use
debug
        wrlong temp, _debugger
:Debug_loop
        nop
        jmp #:Debug_Loop
'-Debug
}


'
'
' Initialized data
'
v_scale                 long pix_ntsc << 12 + frame_ntsc
next_line               long 0

m8                      long    8_000_000
m128                    long    128_000_000
d0                      long    1 << 9 << 0
d6                      long    1 << 9 << 6
d0s1                    long    1 << 9 << 0 + 1 << 1
interlace               long    0
invisible               long    1
visible                 long    2
phaseflip               long    $00000000
phasemask               long    $F0F0F0F0
line                    long    $00060000
lineinc                 long    $10000000
pins0                   long    %11110000_01110000_00001111_00000111
pins1                   long    %11111111_11110111_01111111_01110111
```

```
                        long    sntsc >> 4 << 12 + sntsc                'sync_scale1
                        long    spal >> 4 << 12 + spal
                        long    67 << 12 + lntsc / 2 - sntsc            'sync_scale2
                        long    79 << 12 + lpal / 2 - spal
                        long    %0101_00000000_01_10101010101010_0101   'sync_normal
                        long    %010101_00000000_01_101010101010_0101
ltabx
'
' Uninitialized data
'
taskptr                 res     1                       'tasks
taskret                 res     1
t1                      res     1
t2                      res     1
m1                      res     1
m2                      res     1

x                       res     1                       'display
y                       res     1
hf                      res     1
hb                      res     1
vf                      res     1
vb                      res     1
hx                      res     1
vx                      res     1
hc2x                    res     1
screen                  res     1
tile                    res     1
pixels                  res     1

hvis                    res     1                       'loaded from word table
hrest                   res     1
hhalf                   res     1
vvis                    res     1
vinv                    res     1
vrep                    res     1
burst                   res     1

fcolor                  res     1                       'loaded from long table
sync_scale1             res     1
sync_scale2             res     1
sync_normal             res     1
'
' Parameter buffer
'
_enable                 res     1       '0/non-0        read-only
_pins                   res     1       '%pppmmmm       read-only
_mode                   res     1       '%ccip          read-only
_Scanline               res     1       '@word          read-only
_BackColor              res     1       '@long          read-only
_ht                     res     1       '1+             read-only
_vt                     res     1       '1+             read-only
_hx                     res     1       '4+             read-only
_vx                     res     1       '1+             read-only
_ho                     res     1       '0+-            read-only
_vo                     res     1       '0+-            read-only
_broadcast              res     1       '0+             read-only
_auralcog               res     1       '0-7            read-only
_DoLines                res     1
_Debugger               res     1
```

\ [date]